CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

# Patrones arquitectónicos modernos

Diseño y Patrones Arquitectónicos · Jesús Ariel González Bonilla · 2026-B · Corte 1 · Unidad 3

Patrones arquitectónicos modernos y decisión de migración

# Su v1 ya muestra dónde duele

**Si mañana los pedidos se multiplican por diez, ¿qué parte de su v1 se rompe primero y cuál cuesta más cambiar?**

- **Lo que se cae con la carga**
- **Lo que obliga a tocar todo**
- **Lo que no se puede probar aislado**

# Lo que podrá hacer al terminar

- **Explicar los seis patrones modernos**
- **Reconocer cada patrón en el código**
- **Razonar consistencia frente a disponibilidad**
- **Decidir a qué patrón migrar**
- **Medir la v2 contra la v1**

# 01

# 1. Repartir y ejecutar el sistema

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

# Dos familias, seis patrones

**Unos deciden cómo se reparte y se ejecuta el sistema; otros, cómo se organiza su lógica.**

## Reparto y ejecución

- Microservicios
- Serverless
- Cloud-native

## Lógica y datos

- Hexagonal
- Event-driven
- CQRS

# Un servicio por capacidad de negocio

**Microservicios: servicios autónomos que se despliegan por separado, cada uno con sus datos y su API (Newman, 2021).**

**Cliente**

**API Gateway**

enruta cada petición

**Pedidos**

su propia base

**Productos**

su propia base

# Cinco fundamentos del microservicio

| Fundamento | Qué exige |
|---|---|
| **Contexto delimitado** | Una capacidad de negocio por servicio |
| **Descentralización** | Datos y tecnología propios |
| **Comunicación por API** | Contratos estables y versionados |
| **Aislamiento de fallos** | Circuit breaker y reintentos |
| **Escalado independiente** | Réplicas solo donde hay carga |

# Pedidos consulta productos por red

JAVA · ORDER SERVICE CON FEIGN

```
@FeignClient(name = "product-service")
interface ProductClient {
    @GetMapping("/products/{id}") Product byId(@PathVariable Long id);
}
Product p = productClient.byId(order.getProductId()); // red, no función
```

# Microservicios: cuándo sí y cuándo no

**Sin fronteras claras, los microservicios producen un monolito distribuido.**

## Conviene

- Varios equipos y ritmos
- Partes con cargas distintas
- Fronteras de dominio claras

## No conviene

- Equipo pequeño
- Dominio todavía difuso
- Despliegue manual

PREGUNTA

# ¿Qué patrón despliega por partes?

A Hexagonal, porque aísla el dominio de los pagos

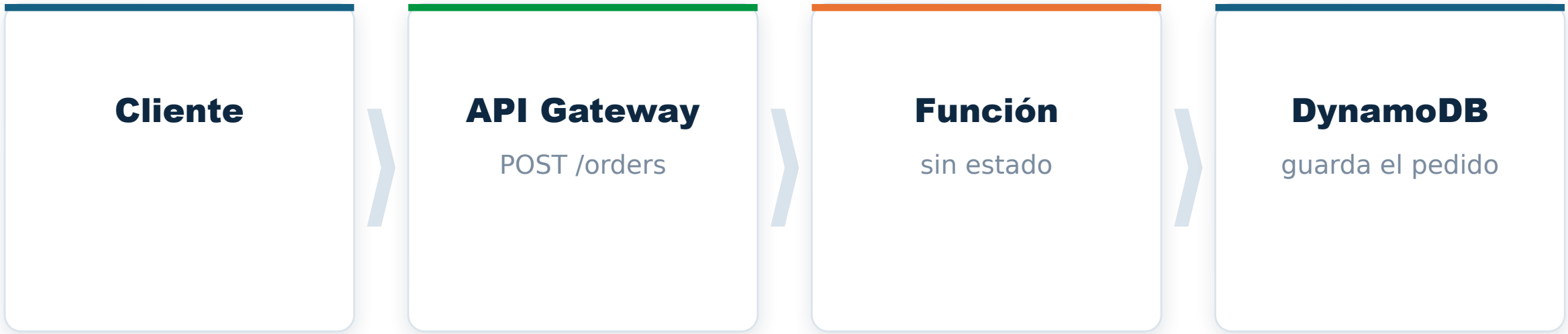
B Microservicios, porque cada servicio se despliega y escala por separado

C CQRS, porque separa las lecturas de las escrituras de pagos

D MVC, porque separa la vista del modelo de pagos

# Serverless: funciones que despiertan

**El proveedor ejecuta la función solo cuando llega un evento y cobra por el tiempo que corre.**



# Una función que registra un pedido

**PYTHON · AWS LAMBDA**

```
table = boto3.resource("dynamodb").Table(os.environ["ORDERS_TABLE"])
def lambda_handler(event, context):
    body = json.loads(event["body"])
    table.put_item(Item={"order_id": str(uuid.uuid4()), **body})
    return {"statusCode": 200, "body": json.dumps(body)}
```

# Lo que serverless gana y lo que cobra

**Brilla con cargas esporádicas; con carga alta y constante puede costar más que un servidor.**

## Gana

- Paga solo por uso
- Escala desde cero
- Sin servidores que parchar

## Cobra

- Arranque en frío
- Máximo 15 min en Lambda
- Atadura al proveedor

PREGUNTA

# ¿Qué es cierto sobre serverless?

A. No hay servidores: el código corre directamente en la red del proveedor

B. Siempre cuesta menos que un servidor dedicado, sin importar la carga

C. Las funciones no guardan estado entre ejecuciones; el estado vive en servicios externos

D. Es la opción adecuada para procesos que duran varias horas

# Cloud-native: hecha para la nube

**No es mudar la aplicación a una máquina en la nube: es diseñarla para contenedores, orquestación y despliegue continuo.**

### Contenedores

Docker empaqueta código y dependencias

### Orquestación

Kubernetes despliega, escala y repara

### Infraestructura como código

Terraform y YAML versionados

### CI/CD

compilar, probar y desplegar en cada cambio

# Tres réplicas y tráfico a las sanas

KUBERNETES · DEPLOYMENT.YAML (FRAGMENTO)

```
spec:
  replicas: 3              # copias fijas; un HPA las autoescala
# dentro de cada contenedor de template.spec.containers:
readinessProbe:            # recibe tráfico solo si responde
  httpGet: { path: /health, port: 80 }
```

# Cloud-native: cuándo sí y cuándo no

**La configuración vive fuera del código: variables de entorno y secretos, nunca la clave en el YAML.**

## Conviene

- Muchos servicios que operar
- Picos de demanda
- Despliegues frecuentes

## No conviene

- Una aplicación pequeña
- Equipo sin Kubernetes
- Sin presupuesto de plataforma

PREGUNTA

# ¿Qué distingue a lo cloud-native?

A. Que usa un proveedor de nube reconocido

B. Que está diseñada para contenedores, orquestación, configuración externa y despliegue automatizado

C. Que corre en una máquina virtual con más memoria

D. Que no necesita base de datos

# 02

# 2. Organizar la lógica y los datos

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

# Hexagonal: el dominio en el centro

**Las dependencias de código apuntan siempre hacia el núcleo (Cockburn, 2005).**

**Adaptadores de entrada · REST, CLI, pruebas automáticas**

**Puertos · interfaces que define el núcleo**

**Dominio · reglas de negocio sin frameworks**

**Adaptadores de salida · memoria, JPA, MongoDB, broker**

# Los puertos los define el núcleo

JAVA · PUERTOS DEL EJEMPLO BANCARIO

```
interface TransferUseCase {                    // puerto de entrada
    void transfer(String fromId, String toId, double amount); }
interface AccountRepository {                  // puerto de salida
    Optional<Account> findById(String id);
    void save(Account account); }
```

# Hexagonal: cuándo sí y cuándo no

**Si el caso de uso se prueba sin base de datos, el hexágono existe; si no, solo hay carpetas.**

## Conviene

- Reglas de negocio complejas
- Integraciones que cambian
- Pruebas rigurosas

## No conviene

- CRUD sin reglas
- Prototipo desechable
- Sin tiempo para interfaces

PREGUNTA

# En la arquitectura hexagonal, ¿hacia dónde apuntan las dependencias de código?

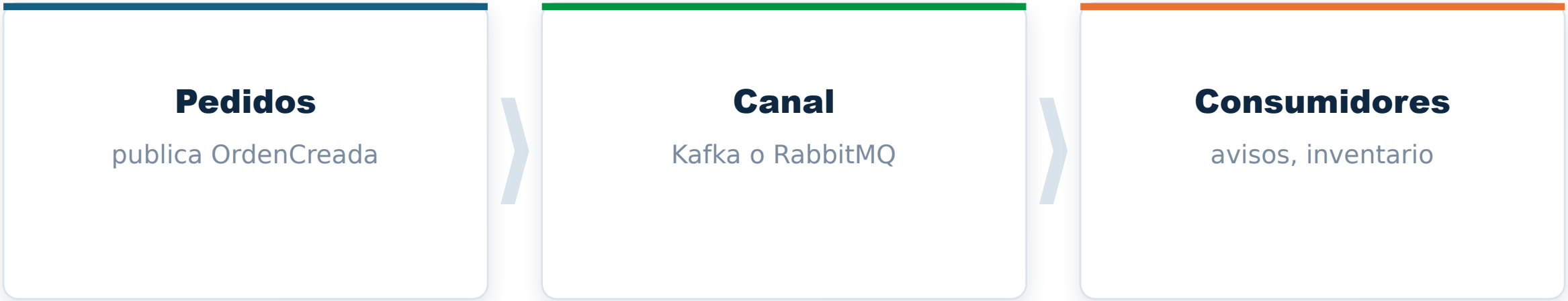
A. Del dominio hacia la base de datos, porque el dominio necesita guardar datos

B. Del dominio hacia el framework web, que es quien recibe las peticiones

C. Hacia el núcleo: el dominio define los puertos y los adaptadores los implementan

D. En ambos sentidos, según el caso de uso

# Event-driven: hechos que viajan




- **Simple: reaccionar a un evento**
- **Streaming: analizar un flujo continuo**
- **Complejo: patrones entre varios eventos**

# Publicar sin conocer al consumidor

**PYTHON · EVENTBROKER (FRAGMENTO)**

```
def subscribe(self, tipo, handler):
    self._subs.setdefault(tipo, []).append(handler)
def publish(self, event):                  # no conoce a los consumidores
    for h in self._subs.get(type(event), []):
        threading.Thread(target=h, args=(event,)).start()  # no bloquea
```

# Un evento es un hecho, no una orden

**Los brokers entregan al menos una vez: los consumidores deben ser idempotentes.**

## Evento

- PedidoConfirmado
- Verbo en pasado
- Cualquiera puede reaccionar

## Orden disfrazada

- EnviarCorreoCliente
- Verbo en imperativo
- Ata al productor

PREGUNTA

# ¿Qué nombre de evento es el correcto?

A EnviarCorreoAlCliente

B ActualizarInventario

C PedidoConfirmado

D ProcesarPago

# CQRS: escribir y leer por separado

**La lectura llega un poco después de la escritura: eso es consistencia eventual (Fowler, 2011).**

**Comando**
crear libro

**Escritura**
PostgreSQL, reglas

**Evento**
LibroCreado

**Lectura**
MongoDB, rápida

# El evento alimenta la lectura

JAVA · LADO DE CONSULTAS

```
@RabbitListener(queues = "book-created-queue")
public void handleBookCreated(BookCreatedEvent e) {
    BookReadModel book = new BookReadModel(e.getId(), e.getTitle());
    repository.save(book);                  // MongoDB, listo para leer
}
```

# CQRS: cuándo sí y cuándo no

**CQRS no obliga a usar dos bases de datos; sí obliga a usar dos modelos.**

## Conviene

- Muchas más lecturas que escrituras
- Consultas con filtros complejos
- Reglas de escritura intrincadas

## No conviene

- Consulta que exige el dato al instante
- CRUD simple
- Equipo sin experiencia distribuida

PREGUNTA

# CitaSalud: ¿dónde cabe CQRS?

A. Pasar la asignación de citas a un modelo de lectura eventual para que responda más rápido

B. Dejar la asignación en el lado de comandos y llevar solo la consulta de agendas a lectura

C. En ninguna parte, porque CQRS siempre exige event sourcing

D. Con una sola base de datos es imposible aplicar CQRS

# 03

# 3. Compromisos: consistencia y CAP

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

# Seis patrones frente a frente

| Patrón | Favorece | Cuesta |
|---|---|---|
| **Microservicios** | Desplegar y escalar por partes | Operación y consistencia |
| **Serverless** | Pagar solo por uso | Arranque en frío, proveedor |
| **Cloud-native** | Elasticidad y autorreparación | Curva y costo de plataforma |
| **Hexagonal** | Probar y cambiar tecnología | Interfaces y mapeos |
| **Event-driven** | Desacoplar en el tiempo | Rastrear el flujo |
| **CQRS** | Lecturas rápidas | Sincronizar dos modelos |

# Los patrones se combinan

### Microservicio + hexagonal

frontera afuera, dominio adentro

### Eventos + CQRS

el evento alimenta la lectura

### Serverless + eventos

una función por evento

### Cloud-native

la plataforma que opera a los demás

# CAP: elegir durante una partición

**Si la red se parte, el sistema elige: responder siempre o no devolver datos viejos.**

## CP

- Espera o rechaza
- Nunca datos viejos
- Citas, cupos, radicados

## AP

- Responde siempre
- Reconcilia después
- Catálogos y consultas

# Cada operación elige su lado

| Operación del reto | Lado | Por qué |
|---|---|---|
| **CitaSalud: asignar cita** | CP | Cero doble asignación |
| **CitaSalud: consultar agenda** | AP | Se corrige al confirmar |
| **RutaSur: bloquear cupo** | CP | Cero sobreventa |
| **AquaBetania: registrar lectura** | AP | Mejor tarde que perdida |
| **TrazaCafé: consulta pública** | AP | Puede tardar unos segundos |

# La saga reemplaza la transacción

1

## Bloquear cupo

transacción local en Reservas

2

## Cobrar

transacción local en Pagos

3

## Confirmar

evento ReservaConfirmada

4

## Si el cobro falla

compensar liberando el cupo

PREGUNTA

# Con la red partida, ¿qué se elige?

A Rendimiento y seguridad

B Consistencia y disponibilidad

C Tolerancia a particiones y escalabilidad

D Nada: en la nube se pueden garantizar las tres propiedades

# 04

# 4. Del foro a la versión 2

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

# Decidir parte del dolor de la v1

1

## Dolor

el límite de la v1, con evidencia

2

## Atributo

el escenario con su medida

3

## Candidatos

dos o tres patrones modernos

4

## Matriz

criterios con peso y puntaje

5

## Costo

lo que se acepta y cómo se mitiga

# Mercado Campesino: la matriz

| Criterio | Peso | Hexagonal y eventos | Microservicios |
|---|---|---|---|
| **Agregar avisos sin tocar el dominio** | 35 | 5 | 4 |
| **Probar el dominio sin base de datos** | 20 | 5 | 3 |
| **Disponibilidad en el cierre del jueves** | 25 | 3 | 4 |
| **Costo de operación para el equipo** | 20 | 4 | 2 |
| **Total ponderado** | 100 | 4,30 | 3,40 |

# El aporte al foro en cinco ejes

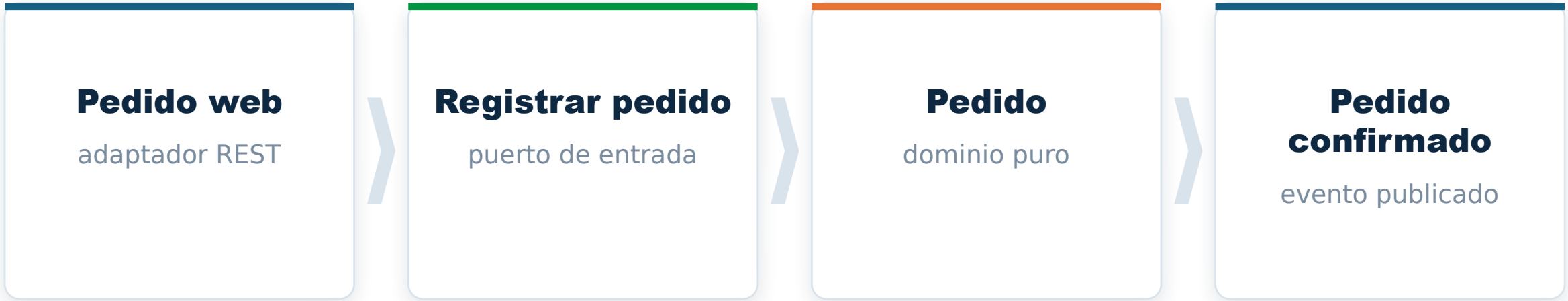
| Eje del foro | Qué escribir sobre su reto |
|---|---|
| **Análisis comparativo** | Los candidatos y su diferencia clave |
| **Escenarios de aplicación** | Qué parte del reto gana con cada uno |
| **Evaluación técnica** | Rendimiento, resiliencia y costos |
| **Integración y evolución** | Nube, DevOps y migración por etapas |
| **Desafíos** | El riesgo principal y su mitigación |

# Seis criterios para medir v1 y v2

**Mismo cambio, misma carga, misma máquina y las dos versiones etiquetadas en git.**

| Criterio | Cómo se mide |
|---|---|
| **Costo de un cambio** | Archivos y líneas tocadas |
| **Acoplamiento** | Dependencias entre módulos |
| **Despliegue independiente** | Sí o no, con evidencia |
| **Rendimiento** | Latencia p95 con la misma carga |
| **Tolerancia a fallos** | Apagar un componente y observar |
| **Facilidad de prueba** | Pruebas del dominio y cobertura |

# Mercado Campesino pasa a hexagonal




- **Avisos y rutas consumen el evento**
- **El cierre del jueves publica PedidosCerrados**
- **Mismo cambio: avisar al productor**

PREGUNTA

# ¿Qué medición v1 frente a v2 vale?

A. Comparar la v2 con un sistema de la industria descrito en un artículo

B. Medir solo la v2, porque la v1 ya se calificó en la Actividad 2

C. Reportar la opinión del equipo sobre cuál versión quedó mejor

D. La misma funcionalidad en v1 y v2, etiquetadas en git, contando archivos y líneas

# 05

# 5. Síntesis de la unidad

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

# Errores que cuestan nota

- Elegir el patrón por moda
- Microservicios que comparten la base
- CQRS donde se exige el dato al instante
- Eventos que son órdenes disfrazadas
- Comparar v1 y v2 en condiciones distintas

# Tres ideas para llevarse

## Cada patrón cobra

ningún beneficio sale gratis

## La operación elige

CP o AP según su regla

## La v1 es la evidencia

la v2 se compara contra ella

# Bibliografía de la unidad


- Newman (2021)
- Richards y Ford (2020)
- Bass, Clements y Kazman (2021)
- Cockburn (2005); Fowler (2011)
- Brewer (2012); Gilbert y Lynch (2002)

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

# ¿Preguntas?

Jesús Ariel González Bonilla · ariel5253@hotmail.com